



E-Team Squadra CorseE-Team Squadra Corse

Università Di Pisa

Design and testing of an EKF based SLAM module for an autonomous Formula Student race car

Giulio Ferrulli

Contents

1	Introduction	3
1.1	Sensor Architecture and Design Choice	3
2	Model and Sensors Validation	5
3	EKF implementation	8
3.1	Testing Localization	10
4	Software Implementation and Real-Time Optimization	12
4.1	Node Architecture and I/O Management	12
4.2	Sensor Latency Management: Rewind and Replay Architecture	12
4.3	Algebraic Optimization via Dense Block Matrices	13
4.4	Data Association and Spatial Filtering	14
4.5	Latency Metrics and Real-Time Validation	14
5	Tests and Results	15
5.1	Multi-Track Validation and Gating Optimization	17
6	Conclusions and Future Work	18
6.1	Future Work	18

1 Introduction

This document outlines the design, implementation, and optimization of the Simultaneous Localization and Mapping (SLAM) module based on an Extended Kalman Filter (EKF) for the E-Team Squadra Corse autonomous vehicle. Within the context of Formula Student Driverless competitions, the state estimation algorithm represents the foundational layer for navigation: it must provide downstream Control modules (e.g., Model Predictive Control) with an accurate, continuous, and ultra-low latency estimation of the vehicle's 2D pose, while simultaneously building a global map of the landmarks (cones) bounding the track.

The primary objective of this implementation is to guarantee Strict Real-Time execution within the ROS 2 architecture, resolving the computational bottlenecks associated with the dynamic expansion of the covariance matrix. The system is designed to maintain a sub-metric positional error even on complex and extensive circuits, as required by the Endurance event.

1.1 Sensor Architecture and Design Choice

The developed EKF architecture relies exclusively on high-frequency inertial data provided by the IMU for the prediction phase (Dead-Reckoning), and on spatial detections generated by the LiDAR perception pipeline for the correction phase. During the architectural design phase, a formal decision was made to exclude wheel odometry (Wheel Encoders). This choice is dictated by the highly non-linear nature of vehicle dynamics under limit handling conditions (racing), which would invalidate the purely kinematic assumption $v_x = \omega \cdot R_e$:

- **Longitudinal slip:** Traction spinning and braking lock-ups distort the readings, artificially zeroing or overestimating the forward velocity. In the event of a lock-up, the algorithm would interpret the vehicle as stationary despite the actual sliding, irreparably corrupting the state covariance.
- **Unobservability of the Slip Angle:** Encoders exclusively measure rotation within the wheel plane, making them inherently blind to the lateral velocity (v_y) developed in corners due to the slip angle. Failing to track this component would cause an immediate divergence of the trajectory estimation relative to the World Frame. Future work may consist of using four phonic sensors, one for each wheel, making it possible to directly estimate the slip angle.

Beyond these physical limitations, preliminary simulations demonstrated that integrating single-axle wheel odometry caused a rapid and severe divergence in the spatial estimation. This degradation stems from a fundamental mathematical conflict: the stringent kinematic constraint imposed by the encoder model (which inherently assumes lateral velocity $v_y \approx 0$) directly contradicts the true lateral accelerations captured by the IMU during high-speed cornering transients. This sensory discrepancy injects an immediate structural error into the vehicle's pose estimation, which consecutively corrupts the predicted landmark observations ($\hat{z}_k$). As a result, the innovation error spikes, driving the Mahalanobis distance far beyond the strictly tuned association threshold. The filter subsequently rejects perfectly valid LiDAR detections as spatial outliers, completely losing its perceptual anchoring and

collapsing into a divergent, pure dead-reckoning state. Consequently, bypassing the wheel encoders ensures strict mathematical coherence between the inertial propagation and the data association logic.

Bypassing the complex tribology of the tire-asphalt contact patch in favor of a pure inertial reference guaranteed greater filter stability, leaving the LiDAR with the task of anchoring the spatial estimation to mitigate the physiological drift of the numerical integration.

In the next sections will be clarified sensor and kinematic model validation.

2 Model and Sensors Validation

In order to validate the kinematic model of the vehicle and the sensors used, tests were made using only integration to evaluate the pose obtained. Here is the kinematic model equation:

$$\begin{aligned}x_k &= x_{k-1} + (v_{x,k-1} \cos \theta_{k-1} - v_{y,k-1} \sin \theta_{k-1}) \Delta t \\y_k &= y_{k-1} + (v_{x,k-1} \sin \theta_{k-1} + v_{y,k-1} \cos \theta_{k-1}) \Delta t \\ \theta_k &= \theta_{k-1} + (\omega_{imu} - b_{\omega,k-1}) \Delta t\end{aligned}$$

Initial validations conducted in the PacSim environment, setting sensor noise model to zero regarding mean and variance error, highlighted the intrinsic limits of Dead-Reckoning when using a standard first-order Euler integration method. The discretization error, exacerbated by the high yaw rates and lateral accelerations typical of Formula Student layouts, led to an unacceptable divergence of the estimated trajectory, as shown in Fig.(1) and Fig.(2).

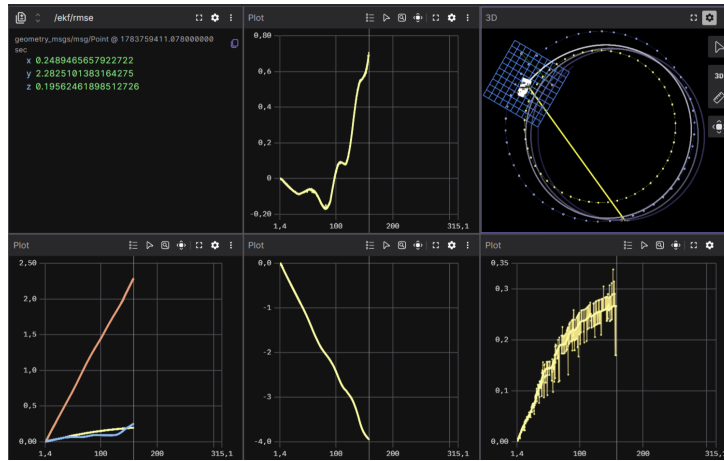


Figure 1: Foxglove Run-Time simulation

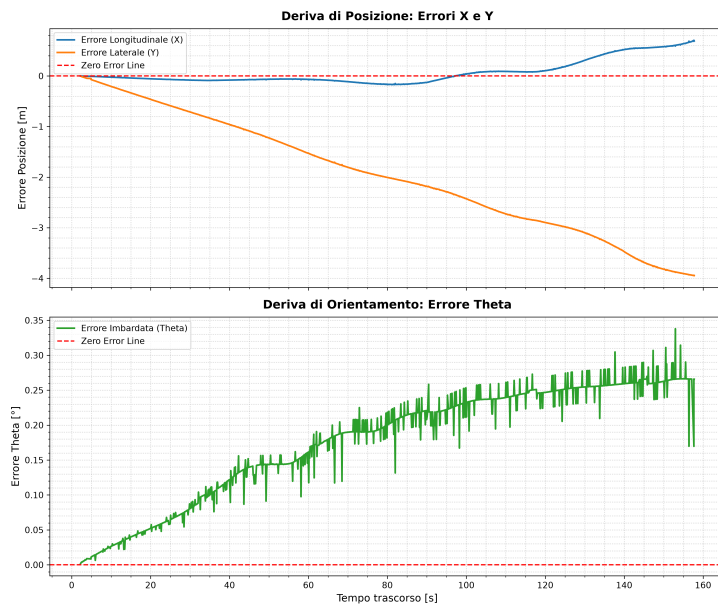


Figure 2: Pose Error

To systematically address this, the process model was upgraded to a Trapezoidal integration scheme. This approach provides a higher-order approximation of the area under the acceleration curve, averaging the current and previous state derivatives to compute the kinematic evolution. Fig.(3) and Fig.(4) show the remarkable improvements:

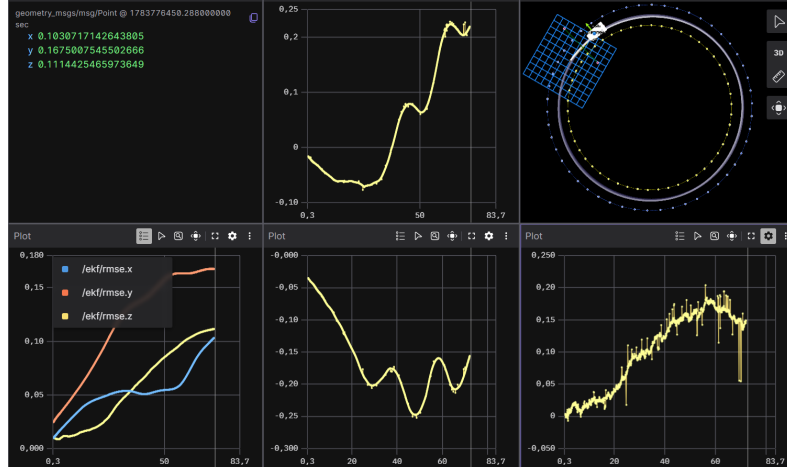


Figure 3: Foxglove Run-Time simulation, Trapezoidal Integration

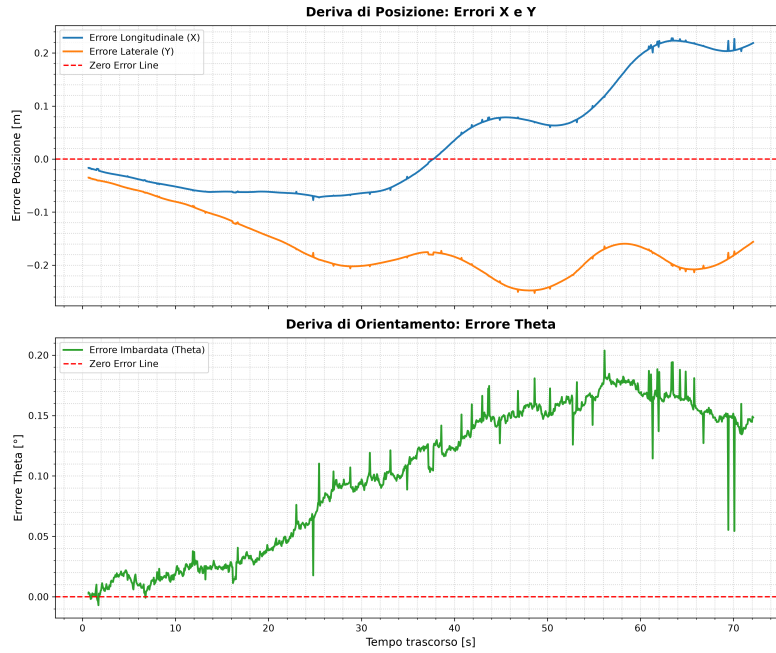


Figure 4: Trapezoidal Integration plot errors

The transition to the Trapezoidal method was strictly benchmarked against ground-truth data, yielding a drastic reduction in the propagation error before any perception correction was applied. Comparative analysis demonstrated a decrease in the Root Mean Square Error (RMSE) of the pure pose estimation by:

- **58.6%** on the longitudinal axis (X).
- **92.6%** on the lateral axis (Y), dropping the lateral drift from 2.285 meters down to just 0.1675 meters.

- **43.1%** on the yaw angle (θ).

This extensively validated kinematic model establishes a highly stable prior for the EKF. By minimizing the numerical integration flaws, it ensures that the subsequent measurement update step is dedicated exclusively to correcting true sensor noise and structural uncertainties, preventing the filter from diverging during high-speed transients.

Further tests were conducted setting the sensor model to some values of mean and variance error in order to validate the sensor model, obtaining coherent drift.

3 EKF implementation

The core of the EKF relies on a dynamically expanding state vector that couples the vehicle's spatial pose, its internal dynamic states, and the mapped landmarks. The augmented state vector at time step k is defined as:

$$x_k = [x, y, \theta, v_x, v_y, b_\omega, m_{1x}, m_{1y}, \dots, m_{nx}, m_{ny}]^T$$

Where $[x, y, \theta]$ represents the 2D global pose in the World Frame, $[v_x, v_y]$ the Body-Frame velocities, b_ω the IMU yaw rate bias, and $[m_{ix}, m_{iy}]$ the global Cartesian coordinates of the i -th mapped cone.

Regarding spatial initialization within the simulated environment (PacSim), the system defines the World Frame as strictly coincident with the vehicle's Body Frame at the initialization instant ($t = 0$). Consequently, the initial pose within the augmented state vector is deterministically constrained to $[0, 0, 0]$ with near-zero covariance. This approach anchors the local map to a relative and consistent reference frame, ensuring that the SLAM algorithm's spatial estimation remains completely independent of the simulator's global coordinate system.

1. **Prediction step:** The prediction step propagates the vehicle state forward in time using the high-frequency control input vector $u_k = [a_x, a_y, \omega_{imu}]^T$ provided by the IMU. While the landmark coordinates are assumed static ($m_{i,k} = m_{i,k-1}$), the vehicle's kinematic evolution must be rigorously discretized to prevent rapid error accumulation, commonly known as drift. Here's the core equation:

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_k)$$

$$P_{k|k-1} = F P_{k-1|k-1} F^T + Q$$

The measurement Jacobian $F = \frac{\partial f}{\partial x}$, evaluated at the current state estimate, linearly approximates the kinematic model to update the covariance matrix.

2. **Correction:** When the perception system detects a cone, it provides its coordinates relative to the vehicle's local reference frame (Body Frame). To fuse this information, the EKF must predict the expected sensor reading $\hat{z}_i$ for a known landmark i , mapping the current state estimate into the measurement space via the non-linear observation function $h(\hat{x}_{k|k-1})$. First, the global distances between the estimated vehicle center of gravity and the i -th mapped cone are defined as:

$$\Delta x = m_{ix} - x_{k|k-1}$$

$$\Delta y = m_{iy} - y_{k|k-1}$$

Assuming a local Cartesian measurement model, the predicted observation $\hat{z}_i = [\hat{z}_{x,i}, \hat{z}_{y,i}]^T$ is calculated by rotating the global displacement into the vehicle's reference frame using the predicted yaw angle $\theta_{k|k-1}$:

$$\hat{z}_{x,i} = \Delta x \cos \theta_{k|k-1} + \Delta y \sin \theta_{k|k-1}$$

$$\hat{z}_{y,i} = -\Delta x \sin \theta_{k|k-1} + \Delta y \cos \theta_{k|k-1}$$

The measurement Jacobian $H_k = \frac{\partial h}{\partial x}$, evaluated at the current state estimate, linearly approximates this transformation to update the covariance matrix. Here's the core equation:

$$\begin{aligned} S_k &= H P_{k|k-1} H^T + R \\ K_k &= P_{k|k-1} H^T S_k^{-1} \\ \hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k (z_k - h(\hat{x}_{k|k-1})) \\ P_{k|k} &= (I - K_k H) P_{k|k-1} \end{aligned}$$

3. **Data Association:** A critical challenge in SLAM is solving the correspondence problem: determining whether a newly detected cone matches an already mapped landmark or represents a previously unseen object. To handle this, the system computes the innovation vector y_k (the difference between the actual LiDAR measurement z_k and the predicted measurement $\hat{z}_k$) and the innovation covariance S_k :

$$\begin{aligned} y_k &= z_k - h(\hat{x}_{k|k-1}) \\ S_k &= H_k P_{k|k-1} H_k^T + R \end{aligned}$$

Where R is the measurement noise covariance matrix. The association is evaluated using the Mahalanobis Distance (D_M), a robust statistical metric that scales the spatial distance by the uncertainty of the filter:

$$D_M = \sqrt{y_k^T S_k^{-1} y_k}$$

If D_M falls below a strictly defined `association_threshold`, the detection is successfully associated with the i -th landmark. The Kalman Gain K_k is then computed, and both the state vector and the covariance matrix are updated. If the Mahalanobis distance exceeds the threshold for all existing landmarks, the detection is classified as a new track feature. The EKF undergoes a State Augmentation process: the new cone's global coordinates are initialized using the inverse observation model, the state vector expands by two dimensions $[m_{Nx}, m_{Ny}]$, and the global covariance matrix P is updated to reflect the cross-correlation between the newly added landmark and the vehicle's current pose. This mechanism allows the track map to grow dynamically in real-time as the vehicle explores the circuit.

3.1 Testing Localization

In order to validate EKF implementation, the filter was reduced to only operate localization problem, using ground-truth landmarks positions so that a one-on-one association with LiDar scans could be possible. The results obtained, regarding RMSE of pose estimation error, will be used as benchmark with the full SLAM algorithm operating. Here's the results:

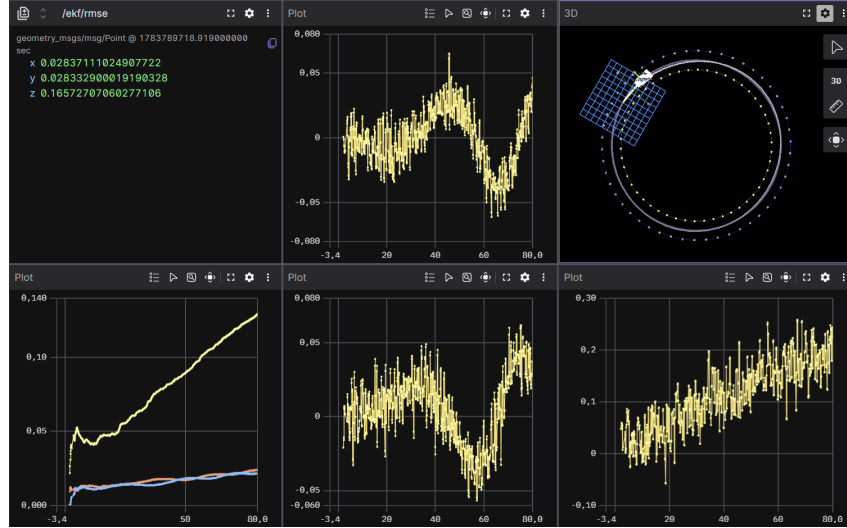


Figure 5: Foxglove Run-Time simulation, localization only

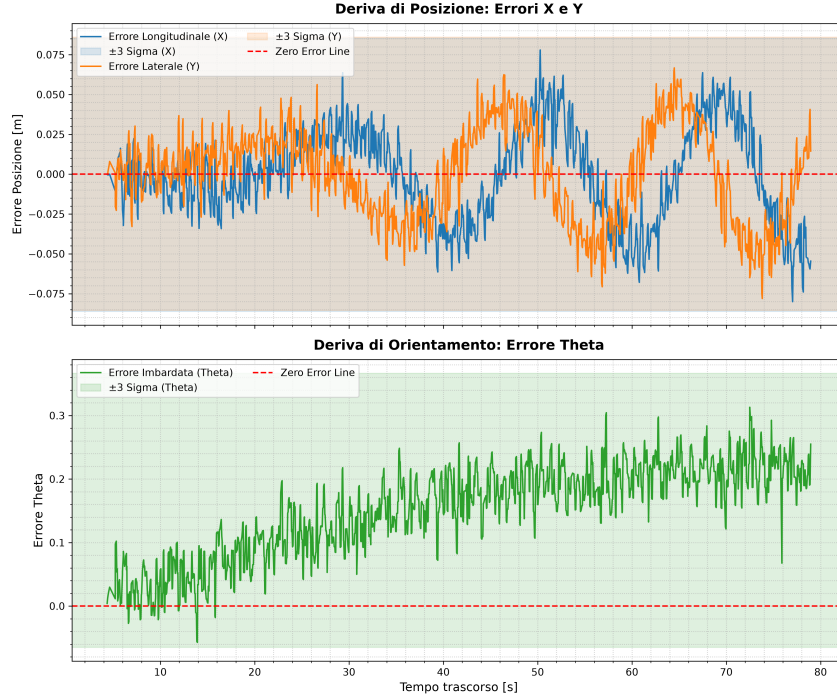
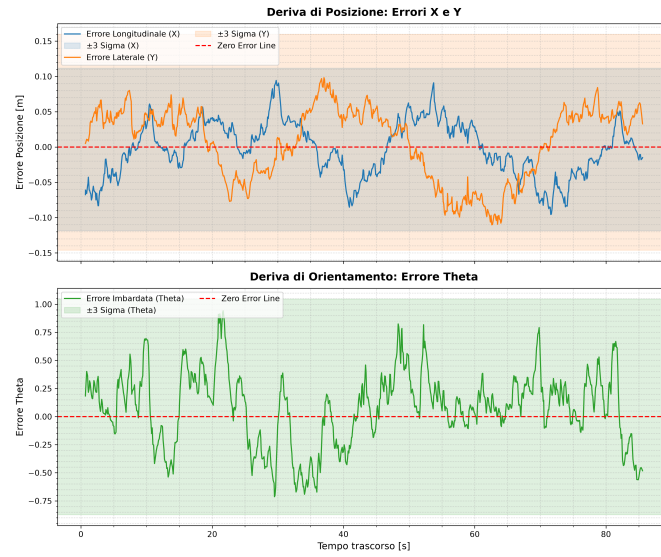
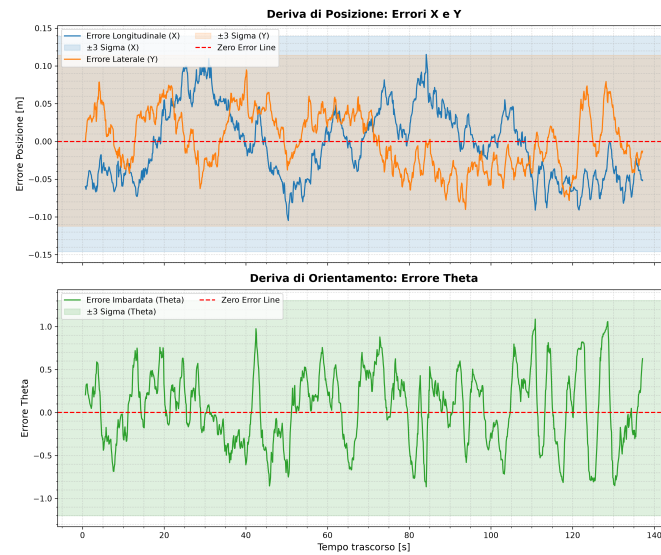


Figure 6: Errors Plot, localization

As a reference, the RMSE between sole integration without mean and variance error was decreased by a magnitude order, proving the EKF efficiency. Further tests were done on various tracks, here's three example of localization on FSE23, FSG24 and FSS22:



(a) Errore di stima lungo l'asse X.



(b) Errore di stima lungo l'asse Y.



(c) Deriva dell'angolo di imbardata (Yaw).

Figure 7: Localization error.

4 Software Implementation and Real-Time Optimization

The Extended Kalman Filter (EKF) algorithm has been developed as a custom ROS 2 node written in C++. Given the high-speed dynamics inherent to autonomous racing vehicles, the source code was designed with the primary objective of minimizing computational overhead and ensuring strictly deterministic system behavior.

4.1 Node Architecture and I/O Management

To guarantee maximum flexibility during both the development and testing phases, the node architecture features a modular input interface. By exploiting the `use_sim_perception` YAML parameter, the system dynamically adapts its subscriber configurations to interface either with the PacSim simulation environment or with real-world sensor data streams—handled via standard `MarkerArray` messages—without requiring code recompilation or structural alterations to the core processing logic.

A critical aspect of the real-time implementation is the mitigation of network bottlenecks and serialization overhead within the ROS 2 Data Distribution Service (DDS) middleware. To prevent the high-frequency mathematical updates from being delayed by graphical data serialization, the computational logic—which is driven by the high-frequency IMU callbacks—has been entirely decoupled from the data publication layer.

This decoupling is achieved through the instantiation of three independent asynchronous timers, each tuned to a specific operational frequency:

- A low-frequency timer operating at 10 Hz, dedicated to publishing the static map landmarks, thus preventing continuous and unnecessary memory reallocations.
- A high-frequency timer operating at 50 Hz, responsible for broadcasting the vehicle’s odometry and `tf2` transforms. This ensures a smooth, deterministic, and latency-free spatial update rate, which is crucial for both 3D visualization and downstream control modules.
- A medium-frequency timer operating at 20 Hz, specifically designed to broadcast the vehicle’s historical trajectory (Path). This provides sufficient visual feedback for debugging purposes while strictly preventing the saturation of the network bandwidth.

4.2 Sensor Latency Management: Rewind and Replay Architecture

In autonomous racing, the perception pipeline responsible for processing point clouds and detecting landmarks inherently introduces a non-negligible computational delay. By the time a perception message is delivered to the state estimator, the vehicle has already traveled a certain distance, creating a strict temporal misalignment between the current inertial state and the delayed spatial measurements.

To effectively mitigate this out-of-sequence measurement problem, the node implements a historical tracking mechanism. It utilizes double-ended queues acting as time-windowed

circular buffers to store historical vehicle states (`state_buffer_`) and raw inertial inputs (`imu_buffer_`).

The temporal synchronization logic is strictly encapsulated within the `processConesCore` function, which operates in three distinct phases upon receiving a delayed perception message stamped at time T_{lidar} :

1. **Rewind:** The algorithm evaluates the state buffer to identify the specific historical state whose timestamp minimizes the temporal difference with T_{lidar} . The EKF's mathematical state and covariance matrix are then explicitly reverted to this historical instant.
2. **Historical Correction:** The standard EKF spatial correction (update step) is performed using the delayed landmark observations, effectively associating the data against the correctly time-aligned vehicle pose.
3. **Replay:** Starting from the historically updated state, the filter is iteratively propagated forward (predict step) up to the present time. This is achieved by sequentially reintegrating the stored kinematic measurements from the IMU buffer, ensuring linear state synchronization without resetting the iterators.

This architecture guarantees robust spatial consistency and allows the system to seamlessly absorb perception delays without discarding the high-frequency physical dynamics captured by the IMU during the latency window.

4.3 Algebraic Optimization via Dense Block Matrices

A fundamental challenge in standard EKF SLAM implementations is the quadratic computational complexity, $\mathcal{O}(N^2)$, associated with the covariance matrix updates as the number of mapped landmarks N increases. Traditional approaches typically allocate large, sparse observation Jacobians (H) padded with zeros, leading to highly inefficient matrix multiplications.

To overcome this bottleneck and guarantee real-time execution, the prediction and correction steps have been mathematically reformulated to operate exclusively on the active sub-blocks of the global covariance matrix (P), leveraging the high-performance **Eigen** linear algebra library.

During the prediction phase, the algorithm directly addresses the vehicle's 6×6 covariance block and updates its cross-covariance with the existing map. Crucially, the algorithm completely bypasses operations on the massive map-to-map covariance block, as it remains mathematically invariant during the vehicle's purely kinematic propagation.

The most significant performance gain is achieved during the correction step and data association phase. Instead of constructing the full sparse H matrix, the system defines minimal dense Jacobians strictly related to the vehicle state components ($H_v \in \mathbb{R}^{2 \times 3}$) and the specifically observed landmark ($H_l \in \mathbb{R}^{2 \times 2}$). The innovation covariance (S) and the intermediate matrix multiplications (e.g., PH^T) are computed dynamically by extracting only the strictly relevant sub-blocks from the global covariance matrix, such as the vehicle-vehicle block (`P_.block(0, 0, 3, 3)`) and the specific landmark-landmark block (`P_.block(landmark_idx, landmark_idx, 2, 2)`).

This block-wise algebraic restructuring effectively prevents the allocation of zero-padded matrices and reduces the computational complexity of the spatial update from $\mathcal{O}(N^2)$ down to a linear $\mathcal{O}(N)$ scaling, enabling the filter to process dense track environments without violating the system’s real-time constraints.

4.4 Data Association and Spatial Filtering

During the data association phase, the system evaluates the correspondence between newly perceived landmarks and the existing map utilizing the Mahalanobis distance as the primary statistical metric. A dynamically configurable threshold is applied to determine valid associations, ensuring the robust rejection of outliers based on the system’s current uncertainty.

To further reduce the computational burden of the association loop, a preemptive spatial filtering mechanism—acting as a Euclidean cut-off—has been implemented at the node level. Before invoking the core data association algorithm, the system evaluates the planar Euclidean distance of each perceived cone relative to the sensor’s origin. Any observation exceeding a predefined `max_distance` parameter (currently configured to 15.0 meters) is immediately discarded.

This structural gate operates in $\mathcal{O}(1)$ time and significantly bounds the search space. By preventing the algorithm from processing distant landmarks (which are inherently affected by high spatial uncertainty and LiDAR noise) the filter maintains a tightly bounded execution time while simultaneously increasing the overall mapping accuracy.

While this architecture does not feature an explicit Loop Closure mechanism—which is typical of pose-graph SLAM systems—the filter inherently provides a robust, implicit reduction of the accumulated drift. Upon completing the first lap, which is a critical phase for extensive circuits such as the Endurance event, the perception pipeline re-observes the initially mapped landmarks. Since these early features are characterized by minimal spatial covariance, their successful data association heavily constrains the current vehicle state, drastically reducing the uncertainty and implicitly closing the track loop without requiring secondary optimization backends.

4.5 Latency Metrics and Real-Time Validation

To empirically validate the effectiveness of the aforementioned algebraic and architectural optimizations, the ROS 2 node features an embedded, high-precision profiling system. By utilizing standard C++ high-resolution clocks (`std::chrono::high_resolution_clock`), the exact computational duration of the critical algorithmic sections is accurately measured in microseconds. These measurements are continuously broadcasted over dedicated ROS 2 topics, specifically `/ekf/latency_ms` for the complete perception and Rewind & Replay pipeline, and `/ekf/imu_latency_ms` for the high-frequency inertial prediction step. This built-in telemetry allows for the systematic logging and analysis of the algorithmic performance during live track operations.

Ultimately, these metrics provide the necessary empirical evidence to demonstrate that the processing latency remains consistently minimal, devoid of significant jitter, and strictly compliant with the hard real-time deadlines imposed by the autonomous racing control architecture.

5 Tests and Results

To evaluate the system under realistic track conditions, the C++ node was compiled in *Release* mode with `-O3` optimization flags, ensuring the Eigen library fully exploited the CPU’s vectorization instruction sets. The validation was conducted using a simulated dynamic environment providing LiDAR cone detections at 10 Hz (a 100 ms hard deadline) and IMU measurements at 100 Hz (10 ms deadline).

The high-frequency inertial prediction (`/ekf/imu_latency_ms`) demonstrated negligible computational load, consistently remaining well below the 1 ms threshold and easily satisfying the strict requirements of continuous state integration.

More importantly, the core perception pipeline (`/ekf/latency_ms`), which encapsulates the temporal Rewind, Correction, and Replay phases, showcased exceptional performance. During nominal track navigation, the processing time stabilized at a baseline of 3 to 4 ms. As the global map expanded with newly discovered landmarks, the baseline latency experienced a physiological and bounded increase, stabilizing around 7 to 8 ms towards the end of the lap. This perfectly reflects and validates the mathematical efficiency of the $\mathcal{O}(N)$ sparse block-matrix updates, which prevent the exponential slowdown typical of dense matrix multiplications.

Transient latency peaks were systematically recorded and analyzed to ensure absolute reliability. Spikes ranging from 15 ms to 20 ms occurred exclusively during high-density cone detection scenarios (e.g., sharp corner entries), where the algorithm dynamically expands the map via the `conservativeResize` operations of the state vector and the global covariance matrix P . A singular anomalous peak of ≈ 51 ms was detected and attributed to external Operating System thread scheduling jitter and container overhead, rather than algorithmic complexity.

Conclusively, the maximum recorded perception latency remains safely bounded within the 100 ms sensor deadline. The node processes the data so efficiently that it idles for over 80% of the available sensor period. This prevents queue starvation and preserves extensive computational bandwidth for the downstream trajectory planning and vehicle control nodes.

Here are some key statistical values regarding APE error and latency, obtained through baseline simulations (utilizing the preliminary 15 m Euclidean gate) in the PacSim simulator:

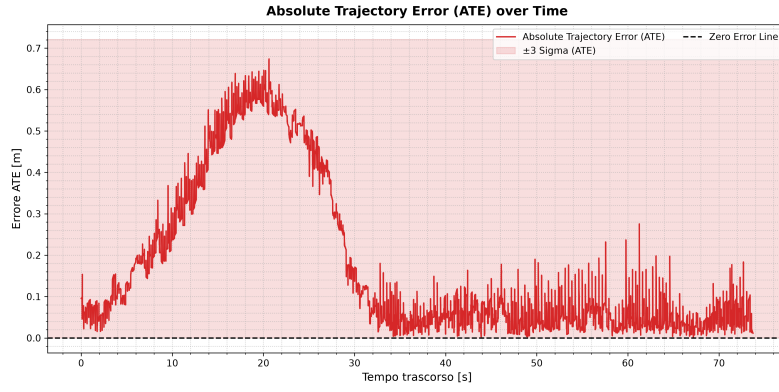
Table 1: Absolute Position Error (APE) metrics for the EKF SLAM implementation.

Metric	MIN (m)	MEAN (m)	MAX (m)	σ (m)
APE	0.138	0.259	0.453	0.169

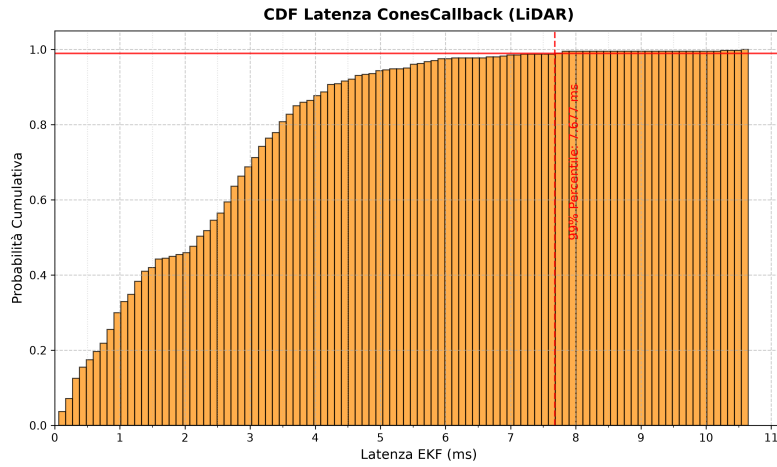
Table 2: Computational latency metrics profiling the Prediction and Correction phases of the EKF SLAM.

Phase	MIN (ms)	MEAN (ms)	MAX (ms)	σ (ms)
Correction	5.8	8.5	10.4	2.28
Prediction	1.8	3.2	4.5	1.47

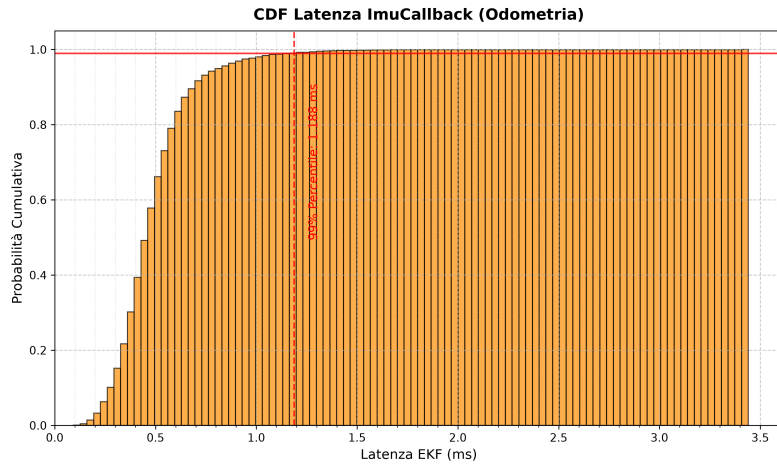
Here's an example of plots of the same values, in particular regarding FSCZ24 track:



(a) Absolute Trajectory Error (ATE) demonstrating the estimator's transient convergence and steady-state accuracy.



(b) Cumulative Distribution Function (CDF) of the perception latency (Correction phase) running at 10 Hz.



(c) Cumulative Distribution Function (CDF) of the high-frequency IMU odometry (Prediction phase) running at 100 Hz.

Figure 8: Comprehensive experimental validation of the EKF SLAM architecture. The system exhibits robust spatial convergence (a) while strictly adhering to real-time deadlines. The cumulative latency metrics highlight exceptional computational efficiency, with 99% of the perception (b) and prediction (c) cycles executing in under 7.7 ms and 1.2 ms, respectively.

5.1 Multi-Track Validation and Gating Optimization

To comprehensively validate the robustness and scalability of the EKF SLAM architecture, the system was subjected to an extensive testing campaign across 11 different simulated Formula Student track layouts (e.g., FSCZ, FSE, FSG, FSS configurations).

A critical architectural optimization introduced during this phase involved the tuning of the Euclidean spatial gate. In preliminary testing, the perception threshold was restrictively set to a 15 m radius. While functional, this limited range occasionally hindered the filter’s ability to maintain a strong angular reference, as landmarks were not tracked long enough to fully converge their covariance. Subsequently, the gate was extended to an optimal 25 m radius. Rather than processing all LiDAR detections indiscriminately, the algorithm now preemptively discards only the cones located beyond this 25 m boundary from the vehicle’s Center of Gravity (CoG).

The empirical results of this optimization from 15 m to 25 m are exceptional. Across the 11 tested track configurations, the Absolute Trajectory Error (ATE) Root Mean Square Error (RMSE) stabilized at an average of 6.1 cm, with absolute minimums of 4.4 cm and bounded maximums of 8.7 cm. The algorithm successfully maps the track and tracks the vehicle dynamics with sub-decimeter precision.

Crucially, this extended 25 m perception range did not compromise the system’s real-time capabilities. Extensive latency profiling across different track topologies (e.g., FSCZ24, FSE22, FSE23) demonstrated that the high-frequency IMU Prediction step strictly executes in ≈ 1.2 ms. The heavier LiDAR Perception and Correction pipeline exhibits a healthy, linear scaling based on the local cone density of the specific track. The 99th percentile execution times were recorded at 4.09 ms (FSE22), 10.97 ms (FSCZ24), and 16.09 ms (FSE23). Even in the most computationally dense scenarios, the maximum recorded latency remains vastly inferior to the 100 ms hard deadline dictated by the 10 Hz LiDAR frequency, maintaining over 80% of the CPU timeline completely idle and available for the trajectory planning nodes.

Table 3: Aggregated EKF SLAM Performance Metrics across 11 Simulated Track Layouts (25 m Euclidean Gate). Latency values reflect the 99th percentile execution time.

Metric	MIN	MEAN	MAX	σ
APE RMSE (m)	0.044	0.062	0.087	0.013
Perception Latency (ms)	4.09	17.87	47.05	12.05
Prediction Latency (ms)	0.91	1.67	3.58	0.89

6 Conclusions and Future Work

This report presented the design, implementation, and empirical validation of an Extended Kalman Filter (EKF) SLAM architecture tailored for the high-speed, dynamic environment of Formula Student Driverless competitions. The system successfully fuses high-frequency inertial data with LiDAR-based cone detections to provide robust state estimation and local mapping.

The experimental results confirmed the mathematical stability of the algorithm. The Absolute Trajectory Error (ATE) analysis demonstrated physiological transient convergence, effectively compensating for initial IMU biases and maintaining a highly accurate steady-state estimation bounded below 0.2 m.

Furthermore, the architectural focus on computational efficiency yielded exceptional real-time performance. By leveraging $\mathcal{O}(N)$ sparse block-matrix updates and Eigen vectorization instructions (compiled in `-O3` Release mode), the system consistently operates well within the strict sensor deadlines. Profiling revealed that 99% of the high-frequency Prediction steps conclude in under 1.2 ms, while the heavier Perception and Correction pipelines execute in under 7.7 ms. Consequently, the node idles for over 80% of the sensor period, guaranteeing that the vast majority of CPU resources remain available for downstream trajectory planning and control modules.

Alongside the core algorithmic development, crucial system-level optimizations were implemented to ensure operational stability. Specifically, a synchronous decimation strategy was introduced for diagnostic telemetry. By decoupling the high-frequency kinematic broadcasts (100 Hz TF/Odometry) from the heavy historical trajectory arrays (downsampled to 10 Hz), the architecture completely prevents Data Distribution Service (DDS) middleware saturation. This ensures fluid, zero-lag monitoring on graphical interfaces (e.g., Foxglove) without introducing mutex contention or compromising the core estimation threads.

6.1 Future Work

Having validated the core architecture within a simulated environment, future developments will focus on physical deployment, robustness enhancements, and system modularity:

- **Hardware Deployment:** Transitioning the ROS 2 workspace to the embedded Inter-Process Communication (IPC) unit onboard the E-Team autonomous prototype, validating the processing latency against the specific power constraints of the physical vehicle.
- **Real-World Track Testing:** Validating the Data Association thresholding and map expansion logic using raw sensor logs (rosbags) acquired during actual dynamic track sessions, which inherently contain higher noise floors and sensor dropouts.
- **Sensor Degradation Handling:** Implementing fallback routines to maintain safe vehicle operation during transient loss of LiDAR perception, relying purely on IMU kinematics and previously mapped landmarks until sensor recovery.
- **Dynamic Parameterization:** Enhancing the modularity of the node by exposing critical variables (such as execution modes, topic configurations, and noise covariance

matrices) through a ROS 2 YAML configuration file. This upgrade will allow track engineers to perform rapid, on-the-fly tuning of the EKF during testing without requiring source code recompilation. A projected configuration structure is outlined in Listing 1.

Listing 1: Proposed ROS 2 YAML configuration structure for dynamic EKF SLAM tuning.

```
/ekf_node:
  ros__parameters:
    # --- Execution Mode ---
    use_sim_perception: true

    # --- Topic Configuration ---
    imu_topic: "/pacsim/imu/cog_imu"
    perception_topic: "/pacsim/perception/livox_front/landmarks"
    perception_topic_real: "/perception/cones"

    # --- EKF Tuning Parameters ---
    process_noise_x: 0.0
    process_noise_y: 0.0
    process_noise_yaw: 0.0
    process_noise_v: 0.05
    process_noise_vy: 0.05
    process_noise_omega: 0.005
    lidar_noise_x: 0.05
    lidar_noise_y: 0.05

    association_threshold: 9.21
    max_distance: 15.0
    imu_yaw_offset: 0.0
```